2026年10月26日

# OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

## OshiruZine-AI

2026年10月26日 号

### 目次


1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. マルチモーダルエージェントの現在地——GUI操作・物理世界へ拡張するAIの実力と課題　*特集（約4000字）*
3. AI規制の世界地図 2026秋——EU AI Act全面施行後の開発現場は何が変わったか　*スタンダード（約2000字）*
4. ローカルLLM 2026秋の選び方——Llama 4・Gemma 3・Phi-4世代のベンチマークと実用Tips　*ショート（500〜1000字）*

# Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

2026年も後半に入り、AIコーディングエージェントはもはや「試してみる」フェーズを完全に脱した。とりわけAnthropicのClaude CodeとOpenAIのCodexは、プロダクション開発の現場で日常的に使われるツールへと成熟し、両者を併用するチームも珍しくなくなっている。しかし、ツールの進化が速すぎるあまり、「なんとなく使っている」開発者と「体系的に使いこなしている」開発者の間には、生産性において数倍の差が生まれつつある。本コラムでは、2026年10月時点での最新動向を踏まえ、これらのAIコーディングエージェントを最大限に活用するためのベストプラクティスを整理する。

## エージェント型開発の現在地

2026年前半、Claude Codeはマルチファイル編集やテスト実行をエージェントとして自律的にこなす能力をさらに強化し、Claude 4 Opus搭載モデルでは数千行規模のリファクタリングを一度のセッションで完遂できるケースが増えた。一方のOpenAI Codexも、コンテナ内でのサンドボックス実行を標準とするアーキテクチャを洗練させ、並列タスク処理の安定性を大幅に向上させている。

注目すべきは、両ツールとも「指示を出して結果を待つ」というシンプルなやり取りから、「計画→実行→検証→修正」のループをエージェント自身が回す設計思想へと明確にシフトしている点だ。開発者の役割は「コードを書く人」から「意図を正確に伝え、出力を検証するアーキテクト」へと変化しつつある。

## CLAUDE.mdとシステムプロンプト設計が成否を分ける

Claude Codeを使いこなすチームが例外なく実践しているのが、`CLAUDE.md`ファイルの戦略的な整備である。このファイルはリポジトリのルートに配置され、プロジェクト固有のコーディング規約、アーキテクチャの方針、禁止パターン、テスト戦略などをClaude Codeに伝える「プロジェクトコンテキスト」として機能する。

2026年現在のベストプラクティスとして、以下の要素を含めることが推奨される。

- **アーキテクチャ概要**: ディレクトリ構成の意図、レイヤー間の依存関係ルール
- **コーディング規約**: 命名規則、エラーハンドリング方針、ロギング基準
- **禁止事項**: 「既存のテストを削除しない」「環境変数をハードコードしない」などの明示的な制約
- **テスト方針**: ユニットテストのカバレッジ基準、テストフレームワークの指定
- **コミットメッセージ規約**: Conventional Commitsの形式指定など

Codex側でも同様に、カスタムインストラクションやリポジトリの`AGENTS.md`に相当する設定ファイルでコンテキストを与える仕組みが整備されている。いずれの場合も、**曖昧さを排除した明文化**が品質を決定的に左右する。

## タスク分割と段階的実行の原則

大規模な機能開発やリファクタリングを一度の指示で丸投げするのは、2026年のモデル性能をもってしてもリスクが高い。現時点でのベストプラクティスは、**1タスク＝1つの明確なゴール**に分割し、各ステップで検証を挟むことだ。

具体的には次のようなワークフローが効果的である。

1. **計画フェーズ**: 「この機能を実装するための計画を立ててください。コードはまだ書かないでください」と明示する

2. **レビューと承認**: 提示された計画をレビューし、方針を確定する
3. **段階的実装**: 計画の各ステップを順番に実行させ、ステップごとにテストを走らせる
4. **統合テスト**: 全ステップ完了後、統合テストとリグレッションテストを実行する

Claude Codeの`--allowedTools`フラグやCodexのサンドボックス設定を活用し、ファイル書き込みやコマンド実行の権限を段階的に与えるアプローチも、意図しない変更を防ぐうえで有効だ。

## テスト駆動でエージェントの出力品質を担保する

AIエージェントが生成するコードの品質を人間のレビューだけで担保するのは、スケーラビリティの観点から限界がある。そこで重要になるのが、**テストをエージェントの「ガードレール」として活用する**考え方だ。

実践的なアプローチとして、先にテストケースを書かせ（またはTDDスタイルで人間が書き）、そのテストを通すコードをエージェントに実装させるフローが高い成功率を示している。Claude Codeは`/test`コマンドでテストの実行と結果のフィードバックループを自動化できるし、Codexもサンドボックス内でのテスト実行結果を踏まえた自己修正が可能だ。

さらに、CIパイプラインとの統合も進んでいる。GitHub ActionsやGitLab CIからエージェントを呼び出し、PRの自動レビューやテスト追加を行わせるワークフローは、多くのチームで標準化されつつある。

## セキュリティとガバナンスの新常識

エージェントがコードを自律的に生成・実行する時代において、セキュリティとガバナンスの重要性はかつてないほど高まっている。2026年後半の時点で押さえるべきポイントは以下の通りだ。

- **シークレット管理**: エージェントに渡すコンテキストにAPIキーやクレデンシャルが含まれないよう、`.claudeignore`や環境変数の分離を徹底する
- **権限の最小化**: エージェントのファイルアクセス範囲やネットワークアクセスを必要最小限に制限する
- **出力の監査**: エージェントが生成した全変更のdiffログを保持し、トレーサビリティを確保する
- **ライセンスコンプライアンス**: 生成コードが既存のOSSライセンスに抵触しないか、SCAツールとの連携で自動チェックする

## 両ツールの使い分けと併用戦略

Claude CodeとCodexの使い分けについて、2026年現在の実務的な指針をまとめると、Claude Codeは長いコンテキストを活かした既存コードベースの理解・リファクタリング・ドキュメント生成に強みがあり、Codexは並列タスクの実行や独立した小機能の高速プロトタイピングに適している。両者をCI/CDパイプラインの異なるステージで併用し、それぞれの強みを引き出すハイブリッド構成を採るチームが増えている。

## まとめ：人間の判断力がますます問われる時代へ

AIコーディングエージェントが高度化するほど、逆説的に「何を作るか」「なぜそう作るか」という人間側の設計判断の質が成果を左右するようになっている。ツールの機能を追いかけるだけでなく、明確なコンテキストの言語化、適切なタスク分割、テストによる品質担保という基本原則を徹底することが、2026年以降のAI駆動開発における最大の競争優位になるだろう。

# マルチモーダルエージェントの現在地——GUI操作・物理世界へ拡張するAIの実力と課題

「AIにブラウザを渡したら、勝手に航空券を予約してきた」——2025年初頭であれば冗談に聞こえたこの一文が、2026年の今、エンジニアの日常風景になりつつある。Claude の Computer Use API がパブリックベータを脱し、Gemini 2.5 が Android／Chrome OS のネイティブ操作権限を獲得し、OpenAI の Operator が企業向け GA となった2026年。マルチモーダルエージェントは「テキストを返すだけの存在」から「画面を見て、クリックし、物理デバイスを動かす存在」へと一気にフェーズを移した。本稿では、この激動の最前線を技術・実装・安全性の3軸で深掘りする。

## LLM が「目」と「手」を持つまでの加速タイムライン

振り返れば、転換点は2024年10月の Anthropic Computer Use ベータだった。スクリーンショットを撮り、座標を指定してマウスとキーボードを操作するというシンプルなアーキテクチャは、しかし業界に衝撃を与えた。翌2025年には Google DeepMind が Project Mariner を Gemini 2.0 に統合し、ブラウザタブ内での自律操作を可能にした。OpenAI も同年1月に Operator を CUA（Computer-Using Agent）として限定公開し、三つ巴の構図が固まった。

2026年に入ってからの進化はさらに急だ。Anthropic は2026年5月に Claude 4 Opus をリリースし、Computer Use API v2 ではマルチディスプレイ対応と「操作意図の自然言語サマリ出力」を標準搭載した。これにより、エージェントが「なぜそのボタンを押したか」を逐次説明できるようになり、監査ログの品質が劇的に上がった。Google は2026年夏の I/O で Gemini 2.5 Ultra のマルチモーダルエージェントモードを発表。Android 端末上でアプリ間遷移をシームレスにこなし、さらに Pixel ロボティクスキットとの連携デモではロボットアームが棚

から商品をピッキングする様子を見せた。OpenAI は Operator v2 で「マルチステップ・プランニングエンジン」を刷新し、1つのタスクを最大50ステップの操作シーケンスに分解して逐次実行・リカバリする能力を示した。

## Computer Use 系 API のアーキテクチャ比較

現在主流の3つのアプローチを整理しよう。

**Anthropic Computer Use API v2** は、スクリーンショットベースの「観測→推論→操作」ループを基本とする。エージェントは一定間隔でスクリーンキャプチャを取得し、VLM（Vision-Language Model）としての Claude がUI要素を認識、座標レベルの操作命令を返す。v2 ではDOM情報の併用（Accessibility Tree の参照）がオプションで追加され、座標認識のロバスト性が大きく向上した。操作はサンドボックス化された仮想マシン上で実行するのが推奨構成であり、Docker コンテナ＋VNC の公式テンプレートが提供されている。

**Google Gemini Agent Mode** は、Chrome DevTools Protocol や Android Accessibility Service と深く統合されている点が特徴だ。OS レベルの API にアクセスするため、スクリーンショット解析だけに頼らず、UI要素のセマンティック情報を直接取得できる。これにより、「このボタンのテキストは『送信』である」という確実な情報を得たうえで操作が可能で、座標ズレによる誤操作が少ない。一方、利用可能なプラットフォームが Google エコシステムに寄っている点はトレードオフとなる。

**OpenAI Operator / CUA v2** は、ブラウザ操作に特化した設計思想で、Chromiumベースの専用ブラウザインスタンス内での操作を前提とする。特筆すべきは「Anticipatory Planning」と呼ばれる機能で、操作前にページ遷移後の予想される画面状態をシミュレーションし、想定外の状態になった場合のフォールバック行動をあらかじめ生成しておく。これにより、ポップアップやCAPTCHA、2要素認証といった中断イベントへの対応力が高い。

三者に共通するのは、「スクリーンショット＋VLMによるUI認識」と「セマンティックなUI情報（DOM/Accessibility Tree）の活用」のハイブリッド化が進ん

でいるという点だ。純粋なスクリーンショットのみに頼るアプローチは、解像度変化やテーマ変更に脆弱であることが実運用で明らかになり、各社ともセマンティック情報との併用へと舵を切っている。

## マルチモーダルエージェントフレームワークの群雄割拠

API だけでなく、オープンソースのフレームワーク群も急成長している。2026年現在、注目すべきプロジェクトを挙げる。

**Browser Use** は、LLM にブラウザ操作を委譲するための Python ライブラリで、GitHub スター数は2026年9月時点で55kを超えた。Playwright をバックエンドに持ち、任意の LLM（Claude、Gemini、GPT、ローカルモデルを含む）をエージェントとしてプラグインできる柔軟性が支持されている。最新版の v2 ではマルチタブの並列操作と、操作履歴からの自動リプレイ・テスト生成機能が追加された。

**LangGraph + Tool Use 拡張** は、LangChain エコシステムの一部として、エージェントのステートマシンを有向グラフで定義し、Computer Use ツールをノードとして組み込む設計パターンを提供する。複雑な業務フロー——例えば「メール確認→添付ファイルダウンロード→Excelで集計→結果をSlackに投稿」といったマルチアプリケーション横断タスク——を宣言的に記述できる。

**Microsoft AutoGen v0.4+** は、マルチエージェント協調に強みを持つ。1つのタスクを「リサーチャーエージェント」「オペレーターエージェント」「レビュアーエージェント」に分担させ、オペレーターの操作結果をレビュアーが検証するというパターンが、本番環境でのエラー率低減に効果を上げている。

これらのフレームワーク共通の設計原則として浮上しているのが、**Human-in-the-Loop（HITL）の段階的緩和** という考え方だ。最初はすべての操作に人間の承認を要求し、信頼性が確認されたサブタスクから徐々に自律実行を許可していく。この「信頼の段階的エスカレーション」は、2026年のエージェント設計におけるベストプラクティスとして広く認知されつつある。

## 物理世界への拡張——ロボティクス連携の最前線

マルチモーダルエージェントのもうひとつの拡張軸が、物理世界への進出だ。Google DeepMind の Gemini Robotics モデル（2025年3月発表、2026年にパートナー企業への限定提供開始）は、Gemini 2.5 のマルチモーダル理解力をロボットの行動計画に直結させた。自然言語の指示を受け、カメラ映像から環境を認識し、ロボットアームの軌道を生成するエンドツーエンドのパイプラインが実現している。

Anthropic も2026年8月に「Claude for Robotics」パートナープログラムを発表し、ROS 2（Robot Operating System 2）との統合レイヤーを提供し始めた。特に注目されるのは、ロボットが操作に失敗した際に「何が起きたか」をビジョン入力から自然言語で説明し、リカバリ戦略を自律的に再計画する能力だ。

ただし、物理世界での動作は GUI 操作とは比較にならないリスクを伴う。ソフトウェア上の誤操作は Ctrl+Z で取り消せるが、ロボットが間違った力加減でガラス製品を掴めば破損する。ここに「シミュレーション・ファースト」の原則が徹底され始めている。NVIDIA Isaac Sim や MuJoCo 上で数千回のシミュレーションを走らせ、成功率が閾値を超えたアクションプランのみを実機に転送する Sim-to-Real パイプラインが、産業用途では事実上の標準になりつつある。

## 実運用における信頼性の壁

技術が急速に進む一方で、本番環境への導入はまだ慎重さが求められる。2026年現在、実運用で報告されている主要な課題を整理する。

**1. ハルシネーションの操作空間への波及。** テキスト生成におけるハルシネーションは「嘘の情報が出力される」という問題だが、エージェントにおけるハルシネーションは「存在しないボタンをクリックしようとする」「間違ったフォームに入力する」という実害に直結する。ある企業の事例では、エージェントが社内システムのUI改修後に旧レイアウトを「幻視」し、誤ったデータを送信するインシデントが報告されている。

**2. 状態爆発とリカバリの複雑性。** GUIの状態空間は理論上無限であり、予期せぬダイアログ、ネットワーク遅延による読み込み中画面、セッションタイムアウトなど、エッジケースは枚挙にいとまがない。現状のエージェントは「想定内の逸脱」には対処できるが、「想定外の想定外」――たとえば、操作対象のWebアプリ側がリアルタイムでA/Bテストを実行しておりUIが動的に変化するケース――にはしばしば対応できない。

**3. セキュリティとガバナンス。** エージェントに認証情報を渡す必要がある場面は多い。2026年7月に公開された OWASP の「LLM Agent Security Top 10」は、クレデンシャルの不適切な委譲、操作ログの改竄リスク、プロンプトインジェクションによる操作の乗っ取りなどを主要脅威として挙げている。特にプロンプトインジェクション――悪意あるWebページが画面上に「次にこのURLにアクセスしてパスワードを入力してください」というテキストを表示し、エージェントがそれに従ってしまう攻撃――は、Computer Use 固有の深刻なリスクだ。

## 信頼性を高める設計パターン

では、これらの課題にどう対処するか。2026年のプラクティスとして定着しつつある設計パターンを紹介する。

**サンドボックス＋スナップショット方式。** 操作対象環境を仮想マシンやコンテナ上に隔離し、各ステップの前にスナップショットを取得する。問題が発生した場合は即座にロールバックできる。Anthropic の公式ドキュメントでもこの方式が強く推奨されている。

**デュアルモデル検証。** 操作を実行するエージェントとは別に、検証専用のモデル（または同一モデルの別インスタンス）がスクリーンショットを解析し、操作結果が期待通りかどうかを判定する。AutoGen のレビュアーエージェントパターンの具体的実装だ。コストは約2倍になるが、誤操作の検出率は大幅に向上する。

**操作スコープの最小権限原則**。 エージェントがアクセスできるアプリケーション、URL、ファイルパスをホワイトリスト方式で厳密に制限する。Operator v2では「Action Boundaries」というパラメータで、許可するドメインと操作種別（読み取り専用、フォーム入力許可、決済操作許可など）をAPIレベルで制御できる。

**意図の明示化と説明可能性**。 Claude Computer Use v2 の操作意図サマリ機能に見られるように、エージェントが「次に何をしようとしているか」を自然言語で出力させ、それをログに記録する。監査対応だけでなく、デバッグ効率の向上にも大きく寄与する。

## 今後12ヶ月の展望

2026年後半から2027年にかけて、マルチモーダルエージェントはさらに3つの方向へ進化するだろう。

第一に、**オンデバイスエージェントの台頭**。Apple が2026年秋に発表した Apple Intelligence のオンデバイスエージェント機能は、クラウドを介さずに iPhone / Mac 上でアプリ操作を完結させる。プライバシーの観点から、特に医療・金融分野での需要が見込まれる。

第二に、**エージェント間通信プロトコルの標準化**。Anthropic が主導する MCP（Model Context Protocol）が2026年にはデファクト化しつつあり、異なるベンダーのエージェント同士が共通プロトコルでツールやコンテキストを共有する世界が近づいている。Google の Agent2Agent Protocol（A2A）との互換性議論も活発だ。

第三に、**規制対応の本格化**。EU AI Act のハイリスク AI 要件がエージェントシステムにも適用される解釈ガイダンスが2026年内に発行される見込みであり、操作ログの保存義務、人間の監督メカニズムの実装、リスクアセスメントの文書化が法的要件となる可能性が高い。

---

マルチモーダルエージェントは、LLMの歴史において「テキスト生成」から「世界への介入」への最大のパラダイムシフトだ。その可能性は圧倒的だが、信頼性・安全性のエンジニアリングなしに本番投入することは、動作テストなしでプロダクションデプロイするようなものである。私たちエンジニアに求められているのは、興奮と冷静さの両方を持って、この新しい道具を正しく使いこなす技術的成熟だろう。2026年は、まさにその成熟が試される年になる。

# AI規制の世界地図 2026秋——EU AI Act全面施行後の開発現場は何が変わったか

「コンプライアンス対応のためにスプリントを1本潰した」——ある欧州向けSaaSを展開するスタートアップのテックリードが、9月のカンファレンスでこぼした一言が印象的だった。2026年8月2日、EU AI Act（人工知能規則）の汎用目的AIモデル（GPAI）に関する義務規定が全面適用となり、開発現場は"規制ファースト"の新しいフェーズに突入している。本コラムでは、各国・地域の規制動向を俯瞰しつつ、エンジニアや研究者が**今すぐ手を動かすべき実務ポイント**を整理する。

## EU AI Act——全面施行で何が変わったか

2024年8月に発効したEU AI Actは、段階的な施行スケジュールに沿って義務が拡大してきた。2025年2月に禁止AIプラクティスの適用が始まり、2025年8月にはGPAIの基本義務（透明性義務・著作権法遵守など）が発効。そして2026年8月、**高リスクAIシステムに関する完全な義務セット**と、**システミックリスクを有するGPAIモデルへの強化義務**が全面適用された。

現場への影響は大きい。具体的には、以下の義務がすべてのGPAI提供者に課されている。

- **モデルカード（技術文書）の整備と当局への提出**：学習データの概要、評価手法、既知の限界を記載
- **EU著作権指令に基づくオプトアウト尊重の立証**
- **下流プロバイダーへの十分な情報提供**

さらに、累積計算量が10^25 FLOPsを超えるなどの基準でシステミックリスクありと分類されたモデルには、**敵対的テスト（レッドチーミング）の実施と報告**、**重大インシデントの欧州AI局への通知義務**が加わる。欧州AI局（AI Office）は2026年夏にGPAI向けの実践規範（Code of Practice）最終版を公表しており、これが事実上の「合格基準」として機能し始めている。

## 米国・中国・日本――三者三様のアプローチ

**米国**は、2025年1月にバイデン政権のAI安全関連の大統領令が撤回されて以降、連邦レベルでは包括的なAI規制法が不在のままだ。一方でカリフォルニア州をはじめ各州法の動きは加速しており、2026年にはコロラド州のAI消費者保護法が施行段階に入った。NISTのAI Risk Management Framework（AI RMF）は任意規格でありながら、訴訟リスクのベンチマークとして存在感を増している。

**中国**は、2023年以降に生成AI管理弁法やディープシンセシス規定など個別規制を積み上げてきたが、2026年にはAIの安全評価制度をさらに拡充。モデルの届出・アルゴリズム登録が実質的な市場参入要件となっている。

**日本**では、2024年4月に策定された「AI事業者ガイドライン」の改定議論が進行中だ。広島AIプロセスの国際指針を踏まえ、2026年度中に高リスク領域でのリスク評価手続きを具体化する方向が示されている。法的拘束力を持つ規制ではなくソフトロー中心のアプローチを維持しつつも、EU AI Actとの相互運用性を意識した設計が注目される。総務省・経済産業省の合同会議では、透明性レポートの推奨フォーマット策定も議題に上がっている。

## 開発チームが今すぐ取り組むべき5つの実務ポイント

各国の規制温度差はあるものの、「透明性」「リスク評価」「文書化」という共通キーワードは明確だ。以下に、規模を問わず開発チームが着手すべきアクションをまとめる。

### 1. モデルカードの"実運用レベル"整備

研究目的の簡易なモデルカードでは不十分になった。EU AI Actの附属書に準拠した技術文書テンプレートを用意し、学習データの出所・フィルタリング方針・評価メトリクス・既知のバイアスを網羅しよう。HuggingFaceが公開しているGPAI向けテンプレートや、欧州AI局のCode of Practiceの附属文書が実務的な出発点になる。

### 2. リスク分類とリスク評価プロセスの内製化

自社のAIシステムがEU AI Actの高リスクカテゴリ（附属書III：採用・信用スコアリング・法執行など）に該当するか判定するフローを確立する。該当しない場合でも、NIST AI RMFの「Map → Measure → Manage → Govern」フレームワークに沿ったリスク評価は、グローバル展開時の共通言語として有効だ。

### 3. 透明性レポートの定期発行

GPAIモデルの提供者には、システミックリスクの有無にかかわらず、下流利用者への情報提供義務がある。四半期または半期ごとの透明性レポートを社内プロセスに組み込み、利用ポリシーの変更・安全性評価結果・インシデント対応履歴を公開する体制を作ろう。

### 4. 著作権コンプライアンスの証跡管理

EU著作権指令のテキスト・データマイニング例外に基づくオプトアウト要請への対応を記録するシステムが必須となった。学習データのクロールログ、robots.txtの解釈ポリシー、オプトアウト要請の受理・対応履歴をトレーサブルに管理する仕組みを整えたい。

### 5. インシデント報告フローの構築

システミックリスクを有するGPAIモデルの場合、重大インシデント発生時には欧州AI局への速やかな報告が義務づけられる。社内のインシデント対応フロー（検知→トリアージ→報告→是正）にAI規制固有のステップを追加し、定期的に机上訓練を実施しておくことを推奨する。

## 規制を"制約"ではなく"設計指針"に

開発者にとって規制対応はコストに見えがちだが、視点を変えれば、モデルカードの整備はドキュメンテーション文化の強化であり、リスク評価の内製化はプロダクト品質の向上そのものだ。EUが先行し、日本・米国が追随する現在の構図は、今後数年で収斂していく可能性が高い。**最も厳しい基準に合わせて設計しておけば、どの市場にも展開できる**——これは多国籍展開するテック企業がすでに採用している戦略でもある。

2026年秋、AI規制の世界地図はかつてないほど詳細に描かれ始めた。次の四半期にどんなルールが追加されても慌てないよう、今日から一つずつ、開発プロセスに規制対応を織り込んでいこう。

# ローカルLLM 2026秋の選び方——Llama 4・Gemma 3・Phi-4世代のベンチマークと実用Tips

「ちょっと試してみるか」——その気軽さで、ノートPCの上に70Bクラスのモデルが立ち上がる時代が来た。2026年に入ってからのオープンウェイトLLMの進

化と量子化技術の成熟は、ローカル推論の実用ラインを一気に押し上げている。この秋、どのモデルをどう選ぶべきか、最新ベンチマークと用途別のポイントを整理したい。

## 主要モデルの現在地

今秋のローカルLLM選定で軸になるのは、**Llama 4（Meta, 2026年春公開）**、**Gemma 3（Google DeepMind）**、**Phi-4（Microsoft）** の3系統だ。いずれも前世代から大幅な改善を遂げている。

- **Llama 4 Maverick（400B MoE / アクティブ約70B）**：多言語・長文脈（最大512K）に強く、MMLU-Proで88超と汎用性が高い。MoE構成のため、量子化との相性が鍵。
- **Gemma 3 27B**：パラメータ効率が際立ち、同サイズ帯でHumanEval+やMBPPのコード系ベンチマークをリード。128Kコンテキストにも対応。
- **Phi-4 14B**：推論特化のチューニングが光り、GPQA-Diamond・MATHなどの推論系ベンチで上位。14Bという軽さでRAGパイプラインに組み込みやすい。

## 量子化技術の進化が支えるローカル実行

llama.cppやMLXの最適化、そして**AQLM/QuIP#系の2〜3bit量子化**が安定期に入ったことで、Llama 4 Maverickでさえ48GB VRAM級のGPU1枚、あるいはApple M4 Max（128GB統合メモリ）で動作する報告が増えている。Q4_K_M量子化であれば、精度低下はMMLUベースで1〜2ポイント程度に収まるケースが多い。「フル精度でなければ使えない」という時代は完全に終わった。

## 用途別・選定ガイド

| 用途 | 推奨モデル | 理由 |
|---|---|---|
| **RAG／社内文書QA** | Phi-4 14B | 軽量で高速。検索結果の統合精度が高く、レイテンシを抑えやすい |
| **コード補完・生成** | Gemma 3 27B | HumanEval+で高スコア。補完の自然さとAPIコード生成の正確性が秀逸 |
| **多言語・長文要約** | Llama 4 Maverick | 512Kコンテキストと多言語性能の両立。日本語精度も前世代から大幅向上 |
| **エッジ・モバイル展開** | Gemma 3 4B / Phi-4-mini 3.8B | 4B以下でスマートフォンやRaspberry Pi 5上での推論に対応 |

## 実用Tips

- **投機的デコーディング（Speculative Decoding）**をドラフトモデル併用で設定すると、体感速度が1.5〜2倍に改善する。Llama 4系では公式にドラフトモデルが配布されている。
- GGUFフォーマットの**imatrix量子化**を選ぶと、単純なラウンド量子化より出力品質が目に見えて向上する。Hugging Face上の「bartowski」「mradermacher」系リポジトリが定番だ。
- vLLMやSGLangでサーブする場合、**PagedAttentionのブロックサイズ調整**が長文脈時のOOMを防ぐ鍵になる。

---

ローカルLLMはもはや「動く」から「選んで使い分ける」フェーズに入った。自分のハードウェアとユースケースに合ったモデル×量子化の組み合わせを見つけ

ることが、この秋の最大のレバレッジポイントだ。まずは手元のマシンで、1モデル動かしてみてほしい。

---